

CampusConnect — Comprehensive Technical Documentation

Part 1 — Complete Project Architecture

CampusConnect is a full-stack, role-based Smart College Management & Collaboration Platform. It connects **Students**, **Faculty**, and **Administrators** through a unified web interface for academic workflows, event registration, coursework submissions, grading, campus notices, grievances, and notifications.

End-to-End Data Flow Architecture



|



client/src/layouts/ (MainLayout vs DashboardLayout)

|



client/src/pages/ (StudentDashboard, FacultyDashboard, AdminDashboard, etc.)

|



client/src/components/ (Navbar, Sidebar, AssignmentCard, EventForm, GradeModal)

|



client/src/services/ (api.js Axios Instance with Authorization: Bearer <JWT>)

|



HTTP REST API Requests (http://localhost:5000/api/...)

|



server/server.js (Express Application & Global Middleware)

|



server/routes/ (authRoutes, assignmentRoutes, eventRoutes, complaintRoutes, etc.)

|



server/middleware/ (authMiddleware -> protect, roleMiddleware -> authorizeRoles)

|



server/controllers/ (assignmentController, eventController, complaintController, etc.)

|



server/models/ (Mongoose Schemas: User, Assignment, Submission, Event, Complaint, Announcement, Notification)

|



MongoDB Atlas (Database Engine & Collections)

|



JSON HTTP Response back to Axios -> React State -> UI Re-render

Part 2 — Client / Frontend Structure

The frontend is built using **React 18**, **Webpack 5**, **Babel 7**, **Tailwind CSS**, and **Lucide Icons**. It resides inside the `client/` folder.

```
client/
├─ index.html          # HTML entry template

├─ package.json        # Dependencies & scripts

├─ webpack.config.js   # Webpack 5 devServer & loader config

├─ .babelrc            # Babel presets config

├─ src/

|   ├─ main.jsx        # React root mount point

|   ├─ App.jsx         # Root React wrapper component

|   ├─ index.css       # Global Tailwind CSS styles

|   ├─ components/     # Reusable UI & feature components

|   ├─ pages/          # Screen views & dashboard pages

|   ├─ context/        # Global React Auth Context

|   ├─ routes/         # Client-side route declarations

|   ├─ layouts/        # Layout shells (Main & Dashboard)

|   ├─ services/       # Axios HTTP service calls

|   ├─ hooks/          # Custom React hooks (useAuth)

|   └─ utils/          # Helper utilities & formatters
```

Folder Explanations

1. components/

- **Why it exists:** Contains reusable UI elements (buttons, inputs, modals, tables) and modular feature components.

- **Subfolders:** `announcements/` , `assignments/` , `complaints/` , `dashboard/` , `events/` .
- **Root Files:** `Navbar.jsx` , `PublicNavbar.jsx` , `Sidebar.jsx` , `Button.jsx` , `Input.jsx` , `Modal.jsx` , `Table.jsx` , `Badge.jsx` , `LoadingSpinner.jsx` , `ProtectedRoute.jsx` , `NotificationDropdown.jsx` .

2. `pages/`

- **Why it exists:** Represents distinct page views rendered at specific URL routes.
- **Files:** `LandingPage.jsx` , `LoginPage.jsx` , `RegisterPage.jsx` , `StudentDashboard.jsx` , `FacultyDashboard.jsx` , `AdminDashboard.jsx` , `ProfilePage.jsx` , `AnalyticsPage.jsx` , `ForbiddenPage.jsx` , `NotFoundPage.jsx` .
- **Subfolders:** `admin/UserManagementPage.jsx` , `announcements/` , `assignments/` , `complaints/` , `events/` , `faculty/StudentPerformancePage.jsx` .

3. `context/`

- **Why it exists:** Provides global state for user authentication.
- **Files:** `AuthContext.jsx` (stores logged-in user details, token, login, logout, and token validation logic).

4. `routes/`

- **Why it exists:** Defines page URLs and role-based route access controls.
- **Files:** `AppRoutes.jsx` (maps paths like `/student/dashboard` , `/faculty/assignments` to specific page components).

5. `layouts/`

- **Why it exists:** Wraps pages inside common headers and sidebars.
- **Files:** `MainLayout.jsx` (PublicNavbar + Footer) and `DashboardLayout.jsx` (Sidebar + Dashboard Header).

6. `services/`

- **Why it exists:** Encapsulates Axios API requests to backend endpoints.
- **Files:** `api.js` (Axios base instance with Bearer token interceptor), `authService.js` , `userService.js` , `assignmentService.js` , `submissionService.js` , `eventService.js` , `announcementService.js` , `complaintService.js` , `notificationService.js` , `dashboardService.js` , `analyticsService.js` , `performanceService.js` , `searchService.js` .

7. `hooks/`

- **Why it exists:** Reusable custom hooks.
- **Files:** `useAuth.js` (shortcut for accessing `AuthContext`).

8. `utils/`

- **Why it exists:** Pure helper functions.
- **Files:** `formatters.js` (date formatting, role badge styling, status pill formatting).

Part 3 — `index.html` Deep Dive

`index.html` is the **single entry HTML page** in our Single Page Application (SPA).

```
<!doctype html>

<html lang="en">

  <head>

    <meta charset="UTF-8" />

    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />

    <meta name="viewport" content="width=device-width, initial-scale=1.0" />

    <title>CampusConnect – Smart College Management Platform</title>

    <link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700;800&display=swap" rel="s

  </head>

  <body class="bg-slate-50 text-slate-900 font-sans antialiased">

    <div id="root"></div>

    <script type="module" src="/src/main.jsx"></script>

  </body>

</html>
```

Key Elements:

1. `<div id="root"></div>` : The placeholder element where React mounts the entire component tree dynamically.
2. `<script type="module" src="/src/main.jsx"></script>` : Tells the browser to load Vite's entry module `main.jsx` , which initializes React.

How it connects to `main.jsx` :

When the browser loads `index.html` , it encounters the script tag pointing to `/src/main.jsx` . Vite executes `main.jsx` , which finds `document.getElementById('root')` and renders `<App />` inside it.

Part 4 — `main.jsx` Deep Dive

`src/main.jsx` is the JavaScript entry point of the React application.

```
import React from 'react'

import ReactDOM from 'react-dom/client'

import App from './App.jsx'

import './index.css'

ReactDOM.createRoot(document.getElementById('root')).render(

  <React.StrictMode>

    <App />

  </React.StrictMode>,

)
```

Line-by-Line Explanation:

- `import ReactDOM from 'react-dom/client'` : Loads React 18's Client DOM rendering engine.
- `import App from './App.jsx'` : Loads the root component of the app.
- `import './index.css'` : Loads global CSS styles (Tailwind utilities & font rules).
- `ReactDOM.createRoot(document.getElementById('root'))` : Selects `<div id="root">` from `index.html` and creates a React rendering root.
- `.render(<React.StrictMode><App /></React.StrictMode>)` : Mounts `<App />` into the DOM.

Part 5 — `App.jsx` Deep Dive

`src/App.jsx` acts as the master wrapper for global providers and client-side routing.

```
import React from 'react';

import { BrowserRouter as Router } from 'react-router-dom';

import { AuthProvider } from '../context/AuthContext';

import AppRoutes from '../routes/AppRoutes';

function App() {

  return (

    <AuthProvider>

    <Router>

    <AppRoutes />

    </Router>

    </AuthProvider>

  );

}

export default App;
```

Responsibility of `App.jsx` :

1. Wraps the app with `AuthProvider` so authentication state is available everywhere.
2. Wraps the app with `BrowserRouter` to listen to URL path changes.
3. Renders `AppRoutes` , which dynamically loads pages based on the route.

Note: `App.jsx` does not import individual cards or forms directly; it delegates route mapping to `AppRoutes.jsx` , which loads pages, which in turn render reusable UI components.

Part 6 — Component Architecture (`src/components/`)

Components in CampusConnect are self-contained, modular UI building blocks.

Core Generic Components:

- `Button.jsx` : Reusable button component supporting `primary` , `secondary` , `outline` , `danger` , and `success` variants.
- `Input.jsx` : Custom form input with error label displays and validation styling.
- `Modal.jsx` : Accessible backdrop overlay for popups, assignment grading, and creation dialogs.
- `Table.jsx` : Data table container used in User Management and Submissions views.
- `Navbar.jsx` : Authenticated header with search bar, unread notification counter, and user profile menu.
- `PublicNavbar.jsx` : Unauthenticated landing page header with public links (Home, Features, About, Sign In, Join).
- `Sidebar.jsx` : Fixed left navigation menu tailored to Student, Faculty, or Admin role links.
- `ProtectedRoute.jsx` : Guard component enforcing JWT presence and matching user roles before rendering child pages.

Feature Component Folders:

- `components/announcements/` : `AnnouncementCard.jsx` , `AnnouncementForm.jsx` , `AnnouncementBadge.jsx` .
- `components/assignments/` : `AssignmentCard.jsx` , `AssignmentForm.jsx` , `SubmissionForm.jsx` , `GradeModal.jsx` .
- `components/complaints/` : `ComplaintCard.jsx` , `ComplaintForm.jsx` , `ComplaintStatusBadge.jsx` .
- `components/events/` : `EventCard.jsx` , `EventForm.jsx` .
- `components/dashboard/` : `DashboardCard.jsx` , `StatCard.jsx` , `RecentActivityList.jsx` .

Part 7 — Pages (`src/pages/`)

Pages represent full-screen views rendered inside layout shells.

Key Pages:

Page Component	Route Path	Access Level	Description
<code>LandingPage.jsx</code>	<code>/</code>	Public	SaaS marketing homepage introducing platform capabilities.
<code>LoginPage.jsx</code>	<code>/login</code>	Public	Form for email/password authentication.
<code>RegisterPage.jsx</code>	<code>/register</code>	Public	New user sign-up with role selection (Student/Faculty).
<code>StudentDashboard.jsx</code>	<code>/student/dashboard</code>	Student	Student overview showing enrolled assignments, registered events, and recent notices.
<code>FacultyDashboard.jsx</code>	<code>/faculty/dashboard</code>	Faculty	Faculty control center for grading, managing assignments, publishing events & announcements.
<code>AdminDashboard.jsx</code>	<code>/admin/dashboard</code>	Admin	Administrative analytics, complaint oversight, and system stats.
<code>ProfilePage.jsx</code>	<code>/profile</code>	Authenticated	View and edit user profile details and change password.
<code>UserManagementPage.jsx</code>	<code>/admin/users</code>	Admin	Admin table to search, activate/deactivate users, and change roles.
<code>AnalyticsPage.jsx</code>	<code>/admin/analytics</code>	Admin	Graphical platform charts (Recharts) for submissions and complaints.

Part 8 — Routing & Protection (`AppRoutes.jsx`)

CampusConnect uses **React Router v6** to define public and protected client routes.

<Routes>

{/ Public Routes /}

<Route element={<MainLayout />>

<Route path="/" element={<LandingPage />} />

<Route path="/login" element={<LoginPage />} />

<Route path="/register" element={<RegisterPage />} />

</Route>

{/ Student Protected Routes /}

<Route element={<ProtectedRoute allowedRoles={['student']} />>

<Route element={<DashboardLayout />>

<Route path="/student/dashboard" element={<StudentDashboard />} />

<Route path="/student/assignments" element={<AssignmentsPage />} />

<Route path="/complaints/create" element={<CreateComplaintPage />} />

</Route>

</Route>

{/ Faculty Protected Routes /}

<Route element={<ProtectedRoute allowedRoles={['faculty']} />>

<Route element={<DashboardLayout />>

<Route path="/faculty/dashboard" element={<FacultyDashboard />} />

```
<Route path="/assignments/create" element={<CreateAssignmentPage />} />

<Route path="/assignments/:id/submissions" element={<AssignmentSubmissionsPage />} />

</Route>

</Route>

{/ Admin Protected Routes /}

<Route element={<ProtectedRoute allowedRoles={['admin']} />>

  <Route element={<DashboardLayout />>

    <Route path="/admin/dashboard" element={<AdminDashboard />} />

    <Route path="/admin/users" element={<UserManagementPage />} />

    <Route path="/admin/analytics" element={<AnalyticsPage />} />

  </Route>

</Route>

</Routes>
```

Part 9 — Authentication Architecture

Authentication is implemented using **JSON Web Tokens (JWT)** and **HTTP Bearer Headers**.

User enters email & password

|

▼

LoginPage calls authService.login(email, password)

|

▼

POST http://localhost:5000/api/auth/login

|

▼

Backend validates user exists & bcrypt.compare(password, user.password)

|

▼

Backend signs JWT with user ID & role (expires in 30 days)

|

▼

Response returned: { token, user: { _id, name, email, role } }

|

▼

AuthContext saves token & user info in localStorage

▼

```
Axios Interceptor in api.js attaches header: Authorization: Bearer <token>
```

▼

```
User redirected to /dashboard (which redirects to /student/dashboard, etc.)
```

Part 10 — Role-Based Access Control (RBAC) Matrix

Feature / Action	Student	Faculty	Admin
View Public Landing Page	✓	✓	✓
View Assignments	✓	✓	✓
Create Assignment	✗	✓	✓
Submit Coursework	✓	✗	✗
Grade Submissions	✗	✓	✓
View Events	✓	✓	✓
Create/Edit Events	✗	✓	✓
Register for Event	✓	✗	✗
Publish Announcements	✗	✓	✓
Raise Grievance / Complaint	✓	✗	✗
Resolve Complaint	✗	✗	✓
System User Management	✗	✗	✓
View Admin Analytics	✗	✗	✓

Part 11 — Frontend Services Layer (`src/services/`)

All API interactions pass through `src/services/api.js`, an Axios instance configured with a base URL (`/api` proxied to `http://localhost:5000`) and request interceptors to auto-attach JWT headers.

Available Services:

- `authService.js` : `login()` , `register()` , `getCurrentUser()` .
 - `assignmentService.js` : `getAssignments()` , `getAssignmentById()` , `createAssignment()` , `updateAssignment()` , `deleteAssignment()` .
 - `submissionService.js` : `submitAssignment()` , `getSubmissionsByAssignment()` , `gradeSubmission()` .
 - `eventService.js` : `getEvents()` , `createEvent()` , `registerForEvent()` , `updateEvent()` , `deleteEvent()` .
 - `announcementService.js` : `getAnnouncements()` , `createAnnouncement()` , `updateAnnouncement()` , `deleteAnnouncement()` .
 - `complaintService.js` : `getComplaints()` , `createComplaint()` , `updateComplaintStatus()` .
 - `notificationService.js` : `getNotifications()` , `getUnreadCount()` , `markAsRead()` , `markAllAsRead()` .
 - `userService.js` : `getUsers()` , `updateUserRole()` , `toggleUserStatus()` , `deleteUser()` .
 - `analyticsService.js` : `getAdminAnalytics()` .
 - `searchService.js` : `performGlobalSearch(query)` .
-

Part 12 & 13 — Backend Architecture (`server/`)

The backend is a Node.js + Express REST API configured in ES Module syntax (`import` / `export`).

server/

```
├─ server.js           # Express application entry point

├─ package.json        # Dependencies (express, mongoose, bcryptjs, jsonwebtoken, cors, dotenv)

├─ config/

|   └─ db.js           # Mongoose MongoDB Atlas connection logic

├─ middleware/

|   └─ authMiddleware.js # JWT verification middleware (protect)

|   └─ roleMiddleware.js # RBAC authorization middleware (authorizeRoles)

|   └─ errorMiddleware.js # Global 404 & 500 JSON error handlers

├─ models/             # Mongoose schemas (User, Assignment, Submission, Event, Complaint, Announcement)

├─ controllers/         # Controller logic for request handling

└─ routes/             # Express router declarations
```

Part 14 — Backend API Routes Table

Method	Endpoint	Description	Auth Required	Role Restrictions
POST	/api/auth/login	Authenticate user & issue JWT	✗	Public
POST	/api/auth/register	Register new Student or Faculty account	✗	Public
GET	/api/users/profile	Fetch logged-in user profile	✓	All Roles
GET	/api/assignments	List assignments	✓	All Roles
POST	/api/assignments	Create new assignment	✓	Faculty, Admin
POST	/api/submissions	Submit assignment work	✓	Student
PUT	/api/submissions/:id/grade	Grade a student submission	✓	Faculty, Admin
GET	/api/events	List all upcoming events	✓	All Roles
POST	/api/events/:id/register	Register student for an event	✓	Student
POST	/api/complaints	Raise a campus complaint	✓	Student
PUT	/api/complaints/:id/status	Update complaint status	✓	Admin
GET	/api/admin/users	List all platform users	✓	Admin
GET	/api/analytics	Fetch admin metrics & counts	✓	Admin

Part 15 — Controllers Layer

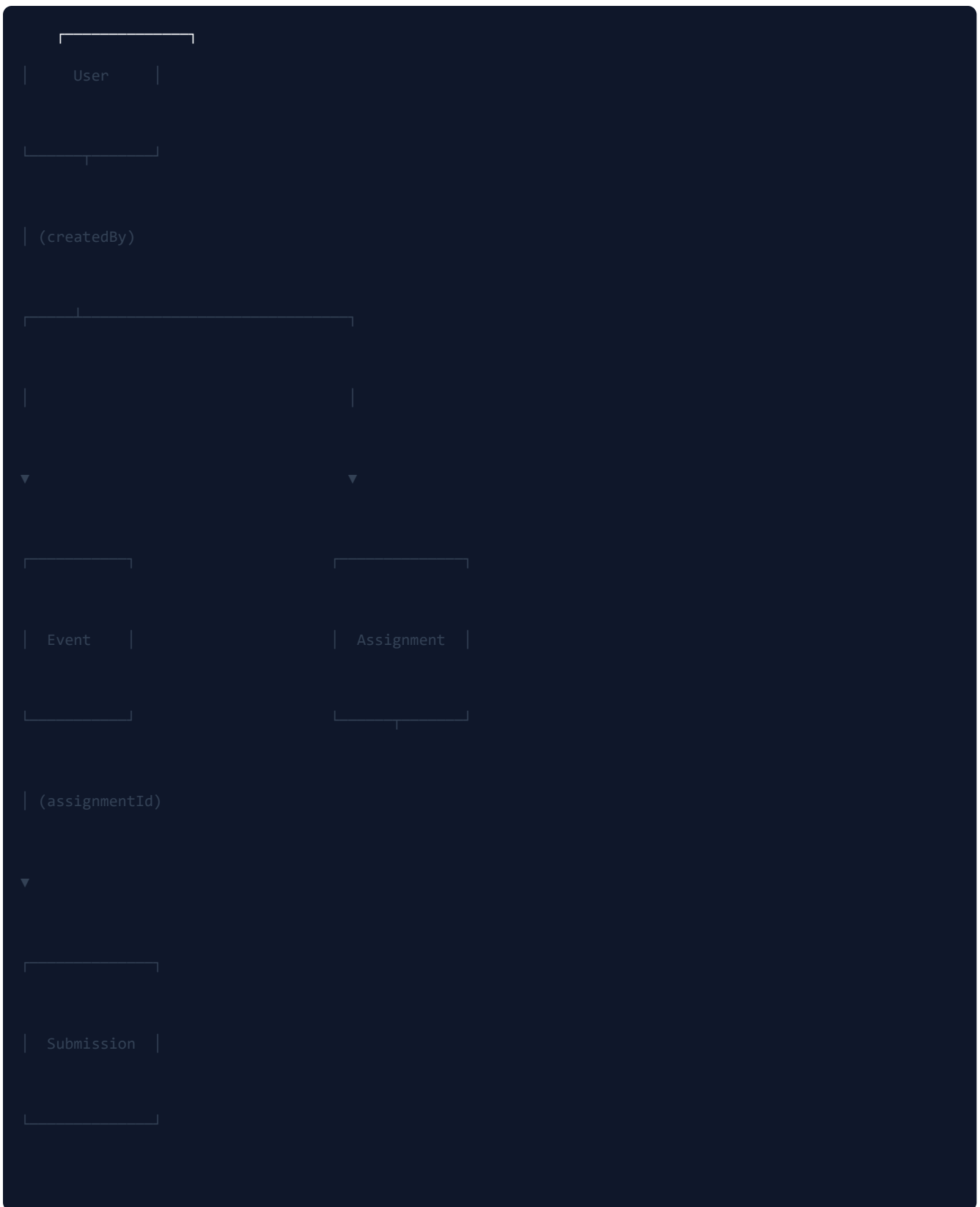
Controllers handle business logic, payload validation, database querying via Mongoose, and returning standardized JSON responses.

Example: `assignmentController.js`

- `getAssignments` : Queries `Assignment.find().populate('createdBy', 'name email')` .
- `createAssignment` : Validates `title` , `subject` , `dueDate` , `totalPoints` , creates document, and emits notifications to students.
- `gradeSubmission` : Updates `Submission` document with `grade` and `feedback` , marks status as `graded` , and sends notification to student.

Part 16 — Database Schemas & Models (`server/models/`)

CampusConnect uses **MongoDB** as its NoSQL database engine via **Mongoose ORM**.



1. **User Schema:** name , email (unique), password (hashed with bcrypt), role (student , faculty , admin), department , isActive , createdAt .
2. **Assignment Schema:** title , description , subject , dueDate , totalPoints , createdBy (Ref: User), targetDepartment , createdAt .
3. **Submission Schema:** assignment (Ref: Assignment), student (Ref: User), submissionText , fileUrl , submittedAt , status (submitted , graded), grade , feedback .
4. **Event Schema:** title , description , category , date , time , venue , organizer (Ref: User), registeredStudents (Array of Ref: User), capacity .

5. **Complaint Schema:** title , description , category , submittedBy (Ref: User), status (pending , in-progress , resolved , rejected), assignedTo (Ref: User), resolutionNotes .
 6. **Announcement Schema:** title , content , category , priority , postedBy (Ref: User), targetAudience , createdAt .
 7. **Notification Schema:** recipient (Ref: User), title , message , type , isRead , createdAt .
-

Part 17 & 18 — Complete Feature Flow Example: Creating & Grading an Assignment

1. Faculty opens Faculty Dashboard and clicks "Create Assignment"

|

▼

2. React renders CreateAssignmentPage.jsx with AssignmentForm.jsx

|

▼

3. Faculty enters Title, Subject, Due Date, Points, and submits form

|

▼

4. Front-end calls `assignmentService.createAssignment(formData)`

|

▼

5. Axios sends POST request to `http://localhost:5000/api/assignments`

with Authorization: Bearer <JWT Token>

|

▼

6. `server.js` routes request through `authMiddleware (protect)`

and `roleMiddleware (authorizeRoles('faculty', 'admin'))`

|



7. `assignmentController.createAssignment` runs:

a. `Validates request body`

b. `Creates Assignment document in MongoDB`

c. `Generates Notification records for enrolled students`

|



8. `Express` returns `201 Created JSON response`

|



9. `React` receives response, updates UI state, and redirects to Assignment list

Part 19 — Frontend vs Backend Comparison

Area	Frontend (<code>client/</code>)	Backend (<code>server/</code>)
Technology	React 18 + Vite	Node.js + Express.js
Styling / Layout	Tailwind CSS	N/A
Main Role	Render UI, manage user interactions	Business logic, authentication, DB operations
State Storage	React State & <code>localStorage</code>	MongoDB Database
Execution Environment	Client Browser	Server / Node runtime
Communication	Sends HTTP requests via Axios	Receives HTTP requests, returns JSON

Part 20 — Key React Concepts Used

1. **Functional Components:** All UI is written as clean React functional components.
 2. **useState** : Handles local state (form inputs, loading states, modal visibility).
 3. **useEffect** : Triggers data fetching from API when components mount or state changes.
 4. **useContext** & **AuthContext** : Manages global logged-in user state without prop-drilling.
 5. **React Router DOM v6:** `Routes` , `Route` , `Navigate` , `useNavigate` , `useLocation` for URL navigation.
 6. **Controlled Components:** Form inputs bound directly to React state values.
-

Part 21 — Key Backend Concepts Used

1. **Node.js & Express:** Fast asynchronous event-driven JavaScript server framework.
 2. **RESTful Routing:** Clean HTTP method mappings (`GET` , `POST` , `PUT` , `DELETE`).
 3. **JWT Authentication:** Stateless authentication token verification in request headers.
 4. **Bcryptjs Hashing:** Secure salted password hashing before saving users to database.
 5. **Mongoose Models:** Schema definitions, validation, and database querying.
 6. **Error Middleware:** Centralized 404 Not Found and 500 Internal Server Error handling.
-

Part 22 — Full Project Directory Tree

CampusConnect/

├─ client/

| └─ index.html

| └─ package.json

| └─ vite.config.js

| └─ tailwind.config.js

| └─ src/

| └─ main.jsx

| └─ App.jsx

| └─ index.css

| └─ components/

| └─ Navbar.jsx

| └─ PublicNavbar.jsx

| └─ Sidebar.jsx

| └─ Button.jsx

| └─ Input.jsx

| └─ Modal.jsx

| └─ Table.jsx

```
| | | ProtectedRoute.jsx
| | |
| | | announcements/
| | |
| | | assignments/
| | |
| | | complaints/
| | |
| | | dashboard/
| | |
| | | events/
| | |
| | pages/
| | |
| | | LandingPage.jsx
| | |
| | | LoginPage.jsx
| | |
| | | RegisterPage.jsx
| | |
| | | StudentDashboard.jsx
| | |
| | | FacultyDashboard.jsx
| | |
| | | AdminDashboard.jsx
| | |
| | | ProfilePage.jsx
| | |
| | | AnalyticsPage.jsx
| | |
| | | admin/
| | |
| | | announcements/
| | |
| | | assignments/
```

```
|   |   ├── complaints/
|   |   ├── events/
|   |   └── faculty/
|   ├── context/
|   |   └── AuthContext.jsx
|   ├── routes/
|   |   └── AppRoutes.jsx
|   ├── layouts/
|   |   ├── MainLayout.jsx
|   |   └── DashboardLayout.jsx
|   ├── services/
|   |   ├── api.js
|   |   ├── authService.js
|   |   ├── assignmentService.js
|   |   ├── eventService.js
|   |   ├── complaintService.js
|   |   └── ...
|   ├── hooks/
```

```
|   |   └─ useAuth.js

|   └─ utils/

|       └─ formatters.js

└─ server/

├─ server.js

├─ package.json

├─ clearDatabase.js

├─ config/

|   └─ db.js

├─ middleware/

|   └─ authMiddleware.js

|   └─ roleMiddleware.js

|   └─ errorMiddleware.js

├─ models/

|   └─ User.js

|   └─ Assignment.js

|   └─ Submission.js

|   └─ Event.js
```

```
|   ├── Complaint.js
|
|   ├── Announcement.js
|
|   └── Notification.js
|
└── controllers/
    |
    |   ├── authController.js
    |
    |   ├── assignmentController.js
    |
    |   ├── eventController.js
    |
    |   ├── complaintController.js
    |
    |   └── ...
    |
    └── routes/
        |
        |   ├── authRoutes.js
        |
        |   ├── assignmentRoutes.js
        |
        |   ├── eventRoutes.js
        |
        |   └── ...
```